CLOUDHELDEN.ORG

IHK AP-2 FIAE - TEIL 1

Prüfungsvorbereitung

Planung eines Softwareproduktes

CLOUDHELDEN.ORG

CLOUDHELDEN.ORG

Einführung

Ausgangssituation vorstellen

Jede Probeklausur enthält eine Ausgangssituation, die einen fiktiven Kontext darstellt.

Aufgaben durcharbeiten

Aufgabenteile 1 bis 4 werden nacheinander vorgestellt. Dabei wird zunächst die Frage vorgestellt, Hinweise gegeben und anschließend die Lösung erklärt.

Kahoot

Zur Festigung der besprochenen Inhalte wird ein Kahoot Quiz durchgespielt.

CLOUDHELDEN.ORG

Ausgangssituation

Die AirCity GmbH betreibt einen regionalen Flughafen und plant die Modernisierung ihres veralteten IT-Systems. Zur Verwaltung des stetig wachsenden Flugbetriebs soll ein neues Software-Paket entwickelt werden. Ihr Team erhält den Auftrag, für diese Organisation diverse Software-Module zu entwickeln.

Softwareentwicklung und Anforderungsmanagement → Algorithmen und Programmierung → Vorgehensmodelle und IT-Sicherheit → UML und Objektorientierte Programmierung

CLOUDHELDEN.ORG

Aufgabe 1: Softwareentwicklung & Anforderungsmanagement

Für jedes der zu entwickelnden Software-Module wird ein Pflichtenheft erstellt. Die darin enthaltenen Anforderungsbeschreibungen müssen anhand der folgenden drei Kriterien gestaltet werden: Vollständigkeit, Eindeutigkeit, Testbarkeit.

Pflichtenheft

Auswahl von Programmiersprachen

CLOUDHELDEN.ORG

1.1 Pflichtenheft: Anforderungen

Ihre Aufgabe:

Beschreiben Sie, welche Konsequenzen sich jeweils für die Entwicklung ergeben, wenn folgende Kriterien bei der Anforderungsbeschreibung nicht beachtet werden:

a) Konsequenzen bei Nichtbeachtung der Vollständigkeit

_____

_____

b) Konsequenzen bei Nichtbeachtung der Testbarkeit

_____

_____

CLOUDHELDEN.ORG

Lösung

Nichtbeachtung der Vollständigkeit

Wenn Anforderungen unvollständig beschrieben sind, werden fehlende Funktionen unter Umständen erst sehr spät im Entwicklungsprozess entdeckt. Es entstehen **teure** Nachbesserungen, **Verzögerungen** im Projektplan und ggf. ein System, das nicht den tatsächlichen **Nutzerbedürfnissen** entspricht.

Vollständigkeit bedeutet, dass alle wichtigen Anforderungen im Pflichtenheft enthalten sind. Also nicht nur grob „das System soll Flüge verwalten“, sondern auch konkrete Details wie:

- Welche Daten zu einem Flug gespeichert werden
- Wer welche Funktionen nutzen darf
- Welche Sonderfälle es gibt
- Welche Regeln gelten, zum Beispiel für Passagieranzahl oder Gate-Zuweisung

Nichtbeachtung der Testbarkeit

Wenn Anforderungen nicht testbar formuliert sind, kann keine sinnvolle **Qualitätssicherung** durchgeführt werden. Die **Qualität** der Software-Module leidet, da unklar bleibt, ob Anforderungen erfüllt wurden – **Fehler** bleiben unentdeckt.

Eine testbare Anforderung ist so formuliert, dass man am Ende klar sagen kann: erfüllt oder nicht erfüllt.

Beispiel für testbar:

„Ein Flugzeug darf maximal 180 Passagiere befördern.“

Das kann man prüfen:

- Bei 180 Passagieren: erlaubt
- Bei 181 Passagieren: nicht erlaubt

Beispiel für nicht testbar:

„Das System soll schnell sein.“

Warum ist das problematisch? Weil „schnell“ jeder anders versteht. Für den einen sind 2 Sekunden schnell, für den anderen nur 0,5 Sekunden.

Hinweis zur Bewertung

Je Teilaufgabe (a und b) wird ein Punkt vergeben. Für teilweise korrekte Beschreibungen kann ebenfalls ein Punkt vergeben werden, sofern der Kerngedanke erkennbar ist. Weiterführende Informationen: Wenn Anforderungen nicht klar formuliert sind, können Missverständnisse entstehen und zu einer höheren Kommunikationslast führen. Für teilweise korrekte Beschreibung wäre ein Punkt zu vergeben.

CLOUDHELDEN.ORG

1.2 Pflichtenheft: Kriterien bewerten

Beurteilen Sie die folgende Anforderung anhand der drei Kriterien Vollständigkeit, Eindeutigkeit und Testbarkeit:

Anforderung:
"Ein Flugzeug darf maximal 180 Passagiere befördern. Aus betrieblichen Gründen darf es jedoch auch mehr sein."

Kriterium	Bewertung mit kurzer Begründung
a) Vollständigkeit	
b) Eindeutigkeit	
c) Testbarkeit	

CLOUDHELDEN.ORG

Anforderung:
"Ein Flugzeug darf maximal 180 Passagiere befördern. Aus betrieblichen Gründen darf es jedoch auch mehr sein."

Lösung

Kriterium	Bewertung	Begründung
a) Vollständigkeit	Vollständig	Die Anzahl der Passagiere ist als Zahlenwert angegeben – relevante Informationen sind vorhanden.
b) Eindeutigkeit	NICHT eindeutig	Der Zusatz "auch mehr sein" macht die Grenze unklar – verschiedene Interpretationen möglich.
c) Testbarkeit	Nicht testbar	Wegen des Widerspruchs kann kein klarer Testfall (Grenzwert) abgeleitet werden.

CLOUDHELDEN.ORG

1.3 Auswahl von Programmiersprachen

Die zu entwickelnde Software-Lösung soll mithilfe einer passenden Programmiersprache erstellt und anschließend effizient und wartbar gestaltet werden.

Ihre Aufgabe:

Nennen Sie vier allgemeingültige Kriterien für die Auswahl einer Programmiersprache.

CLOUDHELDEN.ORG

Lösung

Allgemeingültige Kriterien für die Auswahl einer Programmiersprache (Auswahl):

Performance / Laufzeitverhalten

Wie schnell arbeitet das Programm?
Für zeitkritische Anwendungen ist das sehr wichtig, etwa bei Echtzeitsystemen. Bei Verwaltungssoftware ist extreme Geschwindigkeit oft nicht das wichtigste Kriterium, aber trotzdem relevant.

Speicherverbrauch

Manche Sprachen verbrauchen mehr RAM als andere. Wenn ein System auf schwacher Hardware laufen soll, ist das wichtig.

Portabilität (plattformübergreifend einsetzbar)

Kann die Software auf verschiedenen Betriebssystemen laufen?
Das ist wichtig, wenn ein System nicht nur auf Windows, sondern auch auf Linux oder macOS laufen soll.

Verfügbare Frameworks/Bibliotheken

Eine Sprache ist dann praktisch, wenn es schon viele erprobte Werkzeuge gibt.
Beispiel: Für Webanwendungen gibt es in Java, Python, JavaScript oder C# viele fertige Frameworks.

Bereits vorhandenes Know-How / verfügbare Fachkräfte

Eine Sprache bringt wenig, wenn niemand sie beherrscht.
Wenn das Team Java gut kann, ist Java oft sinnvoller als eine exotische Sprache, auch wenn diese theoretisch Vorteile hätte.

Skalierbarkeit

Code soll später weiterentwickelt werden können. Manche Sprachen fördern eine klare Struktur stärker als andere.

Einsatz von integrierten Entwicklungsumgebungen (IDE-Unterstützung)

Kann man später Entwickler finden, die damit arbeiten können?

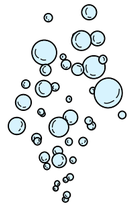
CLOUDHELDEN.ORG

Aufgabe 2: Algorithmen & Implementierung

Bei der Verwaltung des Flughafenbetriebs (Flugdaten, Passagierlisten, Gate-Belegungen) fallen große Datenmengen an, die effizient verarbeitet werden müssen.

Programmgestaltung

Sortieralgorithmen



Bubble-Sort-Algorithmus

CLOUDHELDEN.ORG

2.1 Programmgestaltung

Ihre Aufgabe:

Nennen Sie vier exemplarische Möglichkeiten, wie ein Programm effizient gestaltet werden kann.

CLOUDHELDEN.ORG

Lösung

Weitere sinnvolle Beispiele:

- passende Datenstrukturen wählen
- nur nötige Daten laden
- doppelte Berechnungen vermeiden
- Caching einsetzen

Möglichkeiten, ein Programm effizient zu gestalten (Auswahl):

Schleifen frühzeitig abbrechen

Wenn ich etwas suche und habe es bereits gefunden, muss ich nicht weiter suchen.

Das spart Laufzeit.

Bibliotheksfunktionen nutzen

Viele Standardbibliotheken sind gut getestet und optimiert.

Sie sind oft schneller und sicherer als selbstgeschriebene Lösungen.

Iterativ statt rekursiv

Rekursion ist manchmal elegant, kann aber mehr Speicher benötigen, weil Funktionsaufrufe auf dem Stack landen. Iterative Lösungen sind oft ressourcenschonender.

Unnötige Prüfungen vermeiden

Wenn wahrscheinliche Fälle zuerst geprüft werden, kann das Programm oft schneller reagieren.

CLOUDHELDEN.ORG

2.2 Sortieralgorithmen

Ihr Team diskutiert, ob für die Sortierung der Flugdaten ein quadratisches oder ein logarithmisches Sortierverfahren zu bevorzugen ist.

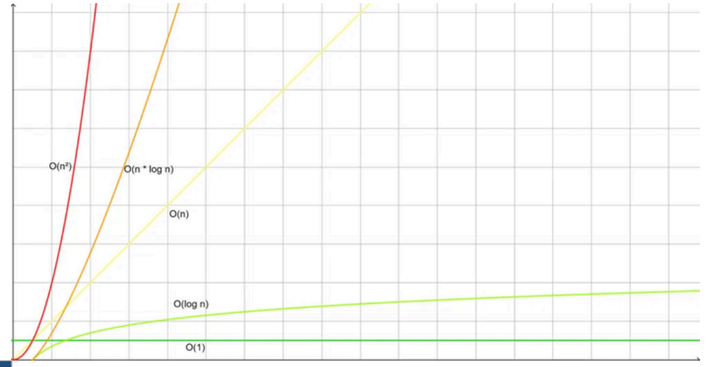
Ihre Aufgabe:

Beschreiben Sie je einen Vorteil, einen Nachteil und geben Sie je einen beispielhaften Sortieralgorithmus für jedes der beiden Verfahren an:

	a) Quadratische Sortierverfahren	b) Logarithmische Sortierverfahren
Vorteil		
Nachteil		
Beispiel		

CLOUDHELDEN.ORG

Lösung



	a) Quadratische Sortierverfahren	b) Logarithmische Sortierverfahren
Vorteil	Einfache Implementierung, gut für kleine Datenmengen geeignet, leicht verständlich, einfaches Debugging möglich.	Effizientere und schnellere Sortierung (O(n log n)), gute Skalierbarkeit, häufig Standard in Bibliotheken.
Nachteil	Je nach Datenmenge sehr ineffizient ($O(n^2)$), viele unnötige Vertauschungen notwendig, nicht optimal für große Datensätze oder verteilte Systeme.	Aufwendigere Implementierung, benötigt ggf. zusätzlichen Speicherplatz, ggf. höherer Verwaltungsaufwand.
Beispiel	Bubble Sort, Selection Sort, Insertion Sort	Merge Sort, Quick-Sort, Heap Sort

Bei einem quadratischen Sortierverfahren wird im schlechtesten Fall jedes Element mit jedem anderen verglichen. Ein solches Verfahren ist dadurch einfach zu programmieren, aber schon ab ca. 100 Elementen kann Bubble Sort ineffizienter sein als Merge Sort. Bubble Sort hat im schlechtesten Fall $O(n^2)$, Merge Sort dagegen besitzt eine stabile Laufzeit von $O(n \log n)$.

CLOUDHELDEN.ORG

2.3 Bubble Sort Algorithmus

Sie möchten in Ihrem Projekt Flug-Objekte sortieren und verwenden hierzu eine Vergleichsfunktion.

Vorgaben:

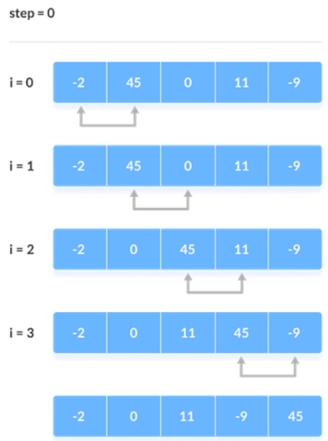
- Es existiert bereits eine Klasse Flug, die Informationen zu einem Flug enthält.
- Die Klasse enthält mehrere Objekte vom Typ Flug in einem Array.
- Die Vergleichsfunktion soll als Funktion-Objekt übergeben werden:
- vergleiche(Flüge[j], Flüge[j+1])
  - Rückgabewert > 0: Flüge[j] hat Vorrang → tauschen;
  - = 0: gleich; < 0: keine Änderung nötig.
- Die Vergleichsfunktion vergleicht Flüge z. B. nach Ziel, Flugdauer, Passagieranzahl und Priorität.

Ihre Aufgabe:

Implementieren Sie den Bubble Sort Algorithmus im Pseudocode unter Verwendung der Vergleichsfunktion.

CLOUDHELDEN.ORG

Lösung



Bubble Sort Algorithmus:

vergleiche(Flüge[j], Flüge[j+1])

Rückgabewert > 0: Flüge[j] hat Vorrang → tauschen;

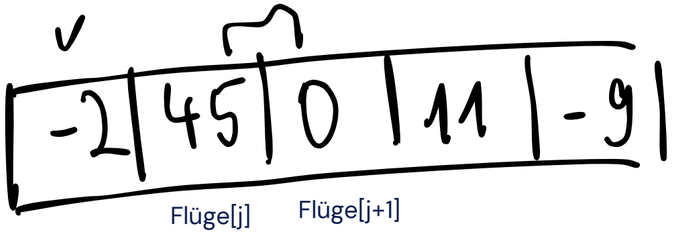
= 0: gleich; < 0: keine Änderung nötig.

```
sort Ueberarray[] Flüge, Function vergleiche : void
    für (i = 0; i < Flüge.length - 1; i++) {
        für (j = 0; j < Flüge.length - 1 - i; j++) {
            wenn (vergleiche(Flüge[j], Flüge[j+1]) > 0) {
                Austausch = Flüge[j]
                Flüge[j] = Flüge[j+1]
                Flüge[j+1] = Austausch
            }
        }
    }
End sort
```

Äußere Schleife

Innere Schleife

Hilfsvariable



Warum Flüge.length – 1 – i?

Weil nach jedem äußeren Durchlauf das größte noch unsortierte Element schon an der richtigen Stelle hinten liegt. Dieser Bereich muss also nicht erneut betrachtet werden.

SWAP:

Hilfsvariablen ← Flüge[j]	Hilfsvariable hat Wert 45
Flüge[j] = Flüge[j+1]	Überschreibe den Wert 45 mit dem Wert 0
Flüge[j+1] = Hilfsvariablen	Überschreibe den Wert 0 mit dem Wert 45

Big-O-Notation

5	**3**	**8**	**1**

4 Elemente im ersten Durchlauf: 3 mal

3 Elemente im 2. Durchlauf: 2 mal

2 Elemente im 3 Durchlauf: 1 mal

1 Elemente im letztem Durchlauf: 2 mal

1. Durchlauf: n-1 Vergleiche

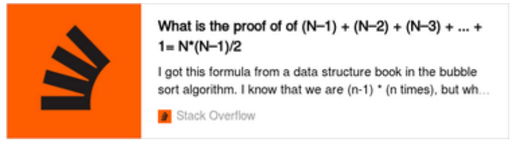
2. Durchlauf: n-2 Vergleiche

3. Durchlauf: n-3 Vergleiche

...

letzten Durchlauf: 1 Vergleich

(n-1) + (n-2) + (n-3) + ... + 1 = n(n-1)/2 $\approx n^2$



CLOUDHELDEN.ORG

Aufgabe 3: Vorgehensmodelle & IT-Sicherheit

Im Rahmen des Projekts stehen wichtige Entscheidungen zu Vorgehensmodellen und zur IT-Sicherheit der webbasierten Komponenten an.

V-Modell und Spiralmodell

Verschlüsselungsarten

SSO

CLOUDHELDEN.ORG

3.1 Unterschied: V-Modell und Spiralmodell

Ihre Aufgabe:

Die Software-Entwicklung erfolgt in einem Umfeld mit möglichen Änderungen und neuen Anforderungen. Um diesen Anforderungen bestmöglich gerecht zu werden, soll die Entwicklung entweder nach dem V-Modell oder nach dem Spiralmodell erfolgen.

Beschreiben Sie den Unterschied zwischen beiden Vorgehensmodellen.

V-Modell = eher linear, Spiralmodell = iterativ und flexibel.

Lösung

V-Modell

Das V-Modell ist stark geplant und phasenorientiert.

Typische Phasen:

- Anforderungsanalyse
- Entwurf
- Implementierung
- Testphasen

Die Besonderheit ist:

- Zu jeder Entwicklungsphase gehört eine passende Testphase.

Beispiel:

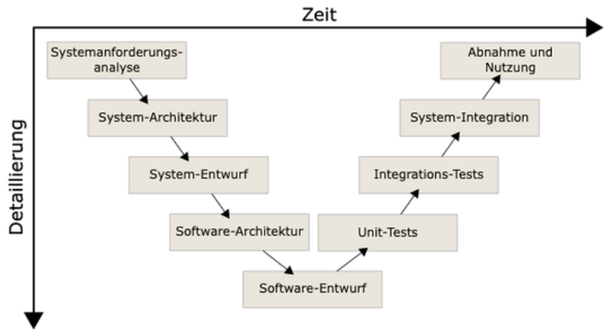
- Anforderungen ↔ Abnahmetest
- Systementwurf ↔ Systemtest
- Moduldesign ↔ Modultest

Stärken:

- klare Struktur
- gute Dokumentation
- hohe Nachvollziehbarkeit
- gut für stabile Anforderungen

Schwächen:

- Änderungen sind aufwendig
- wenig flexibel bei neuen Erkenntnissen



Spiralmodell

Das Spiralmodell arbeitet in wiederholten Schleifen.

Jede Schleife besteht typischerweise aus:

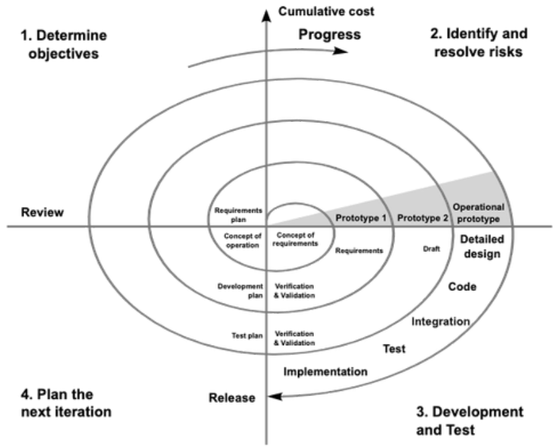
- Planung
- Risikoanalyse
- Entwicklung
- Bewertung
- Man entwickelt also nicht alles starr von Anfang bis Ende, sondern Schritt für Schritt.

Stärken:

- gut bei unsicheren oder wechselnden Anforderungen
- Risiken können früh erkannt werden
- regelmäßiges Feedback möglich

Schwächen:

- aufwendiger in Planung und Steuerung
- mehr Management nötig
- für sehr kleine Projekte oft zu komplex



CLOUDHELDEN.ORG

3.2 Vorgehensmodell wählen

Ihre Aufgabe:

Bekannt aus der vorherigen Aufgabe: Die Software-Entwicklung erfolgt in einem Umfeld mit möglichen Änderungen und neuen Anforderungen. Um diesen Anforderungen bestmöglich gerecht zu werden, soll die Entwicklung entweder nach dem V-Modell oder nach dem Spiralmodell erfolgen.

Entscheiden Sie sich für eines der beiden Vorgehensmodelle und begründen Sie Ihre Wahl.

CLOUDHELDEN.ORG

Lösung

Begründete Entscheidung:

Es sollte das Spiralmodell gewählt werden, da es sich in einem Umfeld mit sich ändernden Anforderungen flexibler anpassen kann (ändernde Rahmenbedingungen).

In der Aufgabenstellung steht, dass mit möglichen **Änderungen** und neuen **Anforderungen** zu rechnen ist.
Genau dafür ist das Spiralmodell geeignet, weil es **flexibel** ist. Neue Erkenntnisse können in der nächsten **Iteration** berücksichtigt werden.
Beim V-Modell wäre das schwieriger, weil dort die Phasen stärker **nacheinander** abgearbeitet werden. Änderungen reißen dann oft frühere Planungen wieder auf.

CLOUDHELDEN.ORG

3.3 Verschlüsselungsarten

Ihre Aufgabe:

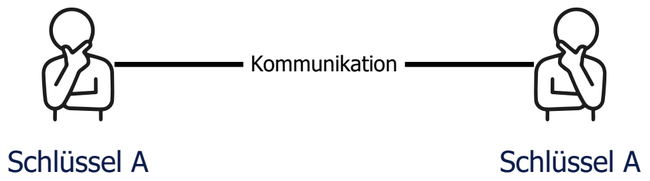
Im Rahmen des Auftrags soll auch ein webbasiertes Portal zur Verwaltung des Flughafenbetriebs bereitgestellt werden. Die Verbindung zum Webserver des Portals muss verschlüsselt erfolgen.

Beschreiben Sie die drei Verschlüsselungsarten und nennen Sie jeweils ein Einsatzgebiet oder Protokoll:

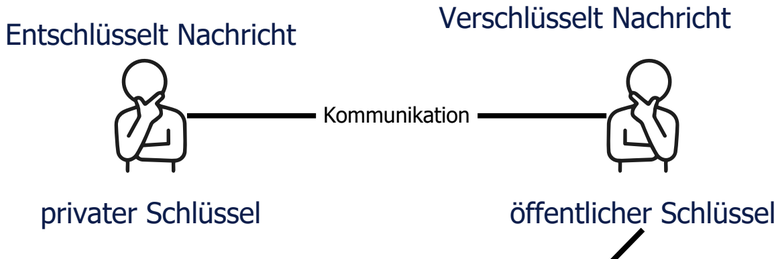
CLOUDHELDEN.ORG

Lösung

	Symmetrische Verschlüsselung	Asymmetrische Verschlüsselung	Hybride Verschlüsselung
Erklärung	• identischer Schlüssel für die Ver- und Entschlüsselung der Daten • Alle Kommunikationsteilnehmer verwenden denselben Schlüssel	• Es wird ein Schlüsselpaar aus einem öffentlichen. und privaten Schlüssel erzeugt • Daten werden mit dem öffentlichen Schlüssel verschlüsselt und können nur mit dem privaten Schlüssel wieder entschlüsselt werden	• Beide Verfahren werden kombiniert • Der Schlüsselaustausch erfolgt asymmetrisch, während die eigentliche Datenverschlüsselung symmetrisch durchgeführt wird
Einsatzgebiete/Protokolle	• Festplatten- oder Dateiverschlüsselung • TrueCrypt-Container • 7-Zip • WLAN-Sicherheit (WPA2), AES, DES	• Nachrichtenverschlüsselung (End-to-End) • Digitale Signaturen • RSA, ECC	• Messenger-Dienste (z.B. Signal) • sichere Übertragung im Internet (HTTPS) • TLS, SSH

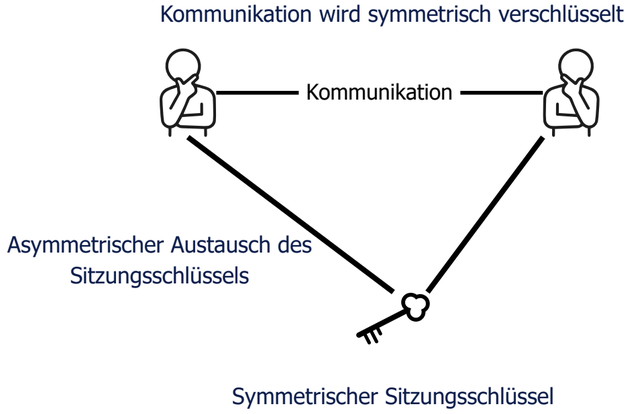


Schlüsselaustauschproblem!



langsamer

darf bekannt werden!



CLOUDHELDEN.ORG

3.4 Single Sign-On (SSO)

Die Nutzer des webbasierten Portals müssen sich anmelden, um auf die jeweiligen Systeme zuzugreifen. Dabei soll im Unternehmen ein vorhandener Identity Provider genutzt werden, der OAuth2 integriert wird.

Ihre Aufgabe:

Beschreiben Sie je einen Vorteil und einen Nachteil von Single Sign-On (SSO) im Hinblick auf die Sicherheit.

CLOUDHELDEN.ORG

Lösung

Vorteile beim Einsatz von SSO

- Durch die **zentralisierte Rechteverwaltung** kann leichter ein ganzheitlicher Überblick über die erteilten Rechte eingesehen werden.
- **Sicherheitsvorfälle** können schnell und wirkungsvoll behoben werden, da nur eine zentrale Stelle angepasst werden muss.
- Nutzer benötigen **keine unterschiedlichen Benutzernamen** und **Passwörter** – die Gefahr, Passwörter aufzuschreiben, sinkt.

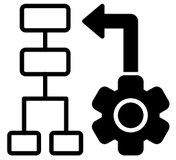
Nachteile beim Einsatz von SSO

- Vollständige Abhängigkeit von der Verfügbarkeit des **Identity Providers** – fällt dieser aus, sind alle Systeme nicht erreichbar.
- Sollte ein **Zugang in die Hände von Angreifer**n gelangen, haben diese ggf. direkten Zugang zu mehreren Systemen. **SPOF**
- Die Spezifikation von OAuth2 empfiehlt ausdrücklich die Verwendung von TLS/SSL, während der Token unverschlüsselt verschickt wird – **Implementierungsfehler** können Schwachstellen erzeugen.

CLOUDHELDEN.ORG

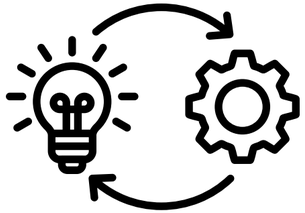
Aufgabe 4: UML & Ojektorientierte Programmierung

Die Anwendung zur Verwaltung von Passagieren und Flügen soll als objektorientiertes System umgesetzt werden.



Sequenzdiagramm

Klassendiagramm



Implementierungen

CLOUDHELDEN.ORG

4.1 Sequenzdiagramm: OAuth-Autorisierung

Ihre Aufgabe:

Bei OAuth handelt es sich inzwischen um den de-facto-Standard der Client-Autorisierung. Vereinfacht wird hierbei folgender Prozess durchlaufen:

Ablauf der OAuth-Autorisierung:

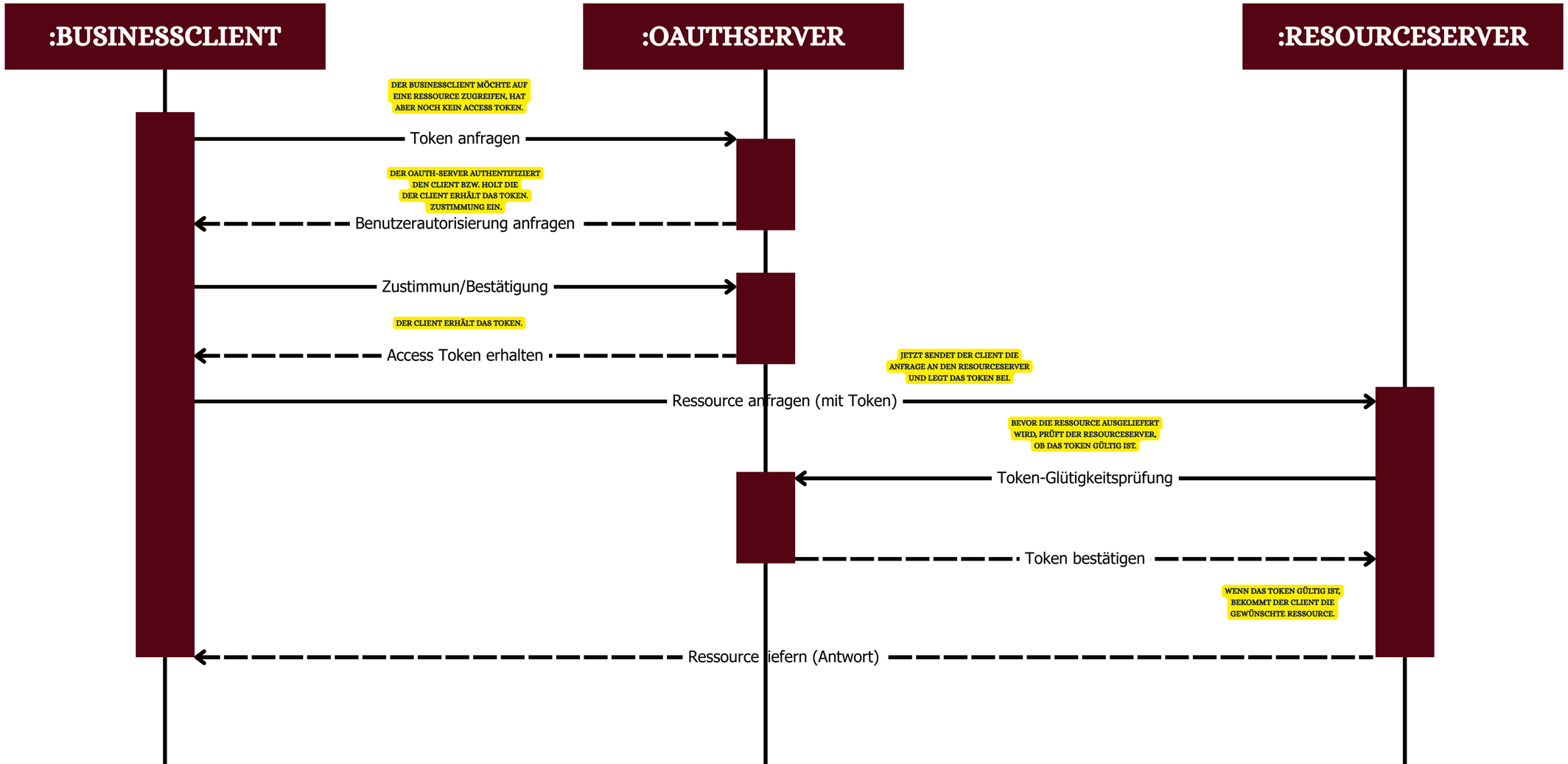
1. Der Client sendet eine Anfrage an einen Autorisierungsserver, um ein Access Token zu erhalten.
2. Der Autorisierungsserver verifiziert den Client und entscheidet anschließend über die Benutzerautorisierung.
3. Der Autorisierungsserver gibt den Zugriff auf den Ressourcen-Server des gewünschten Clients frei.
4. Der Server prüft, bevor er die Ressource an den Client liefert, ob das Token beim Autorisierungsserver gültig ist; dieser antwortet mit einem entsprechenden Ergebniscode.

CLOUDHELDEN.ORG

Ablauf der OAuth-Autorisierung:

1. Der Client sendet eine Anfrage an einen Autorisierungsserver, um ein Access Token zu erhalten.
2. Der Autorisierungsserver verifiziert den Client und entscheidet anschließend über die Benutzerautorisierung.
3. Der Autorisierungsserver gibt den Zugriff auf den Ressourcen-Server des gewünschten Clients frei.
4. Der Server prüft, bevor er die Ressource an den Client liefert, ob das Token beim Autorisierungsserver gültig ist; dieser antwortet mit einem entsprechenden Ergebniscode.

Lösung



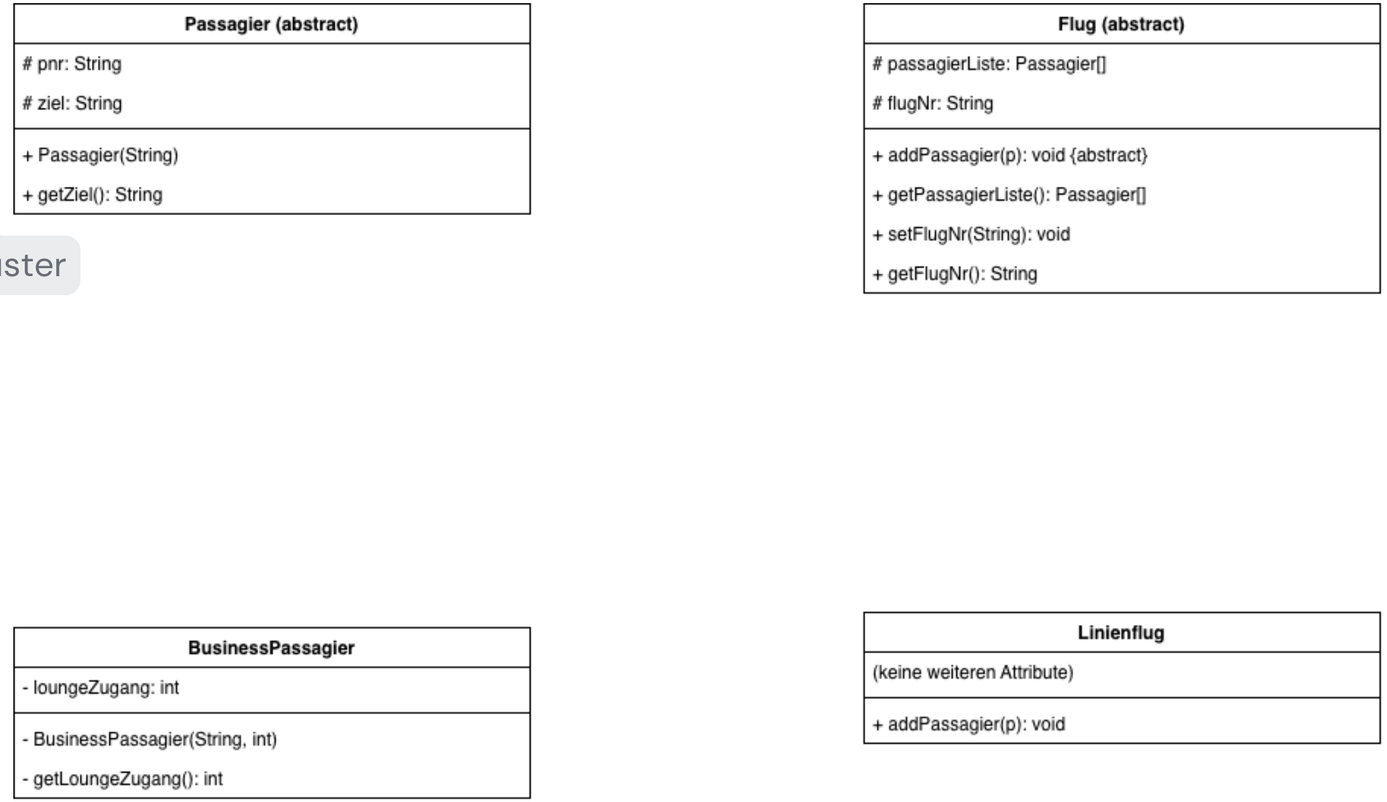
CLOUDHELDEN.ORG

4.2 Klassendiagramm: Beziehungen ergänzen

Ihre Aufgabe:

Die Anwendung zur Verwaltung der Passagiere soll eine objektorientierte Struktur nach dem untenstehenden Muster erhalten.

Nennen Sie die fehlenden Beziehungen im folgenden Klassendiagramm. Hinweis: Dieses Beispiel wird in den nachfolgenden Aufgaben weiterverwendet.



CLOUDHELDEN.ORG

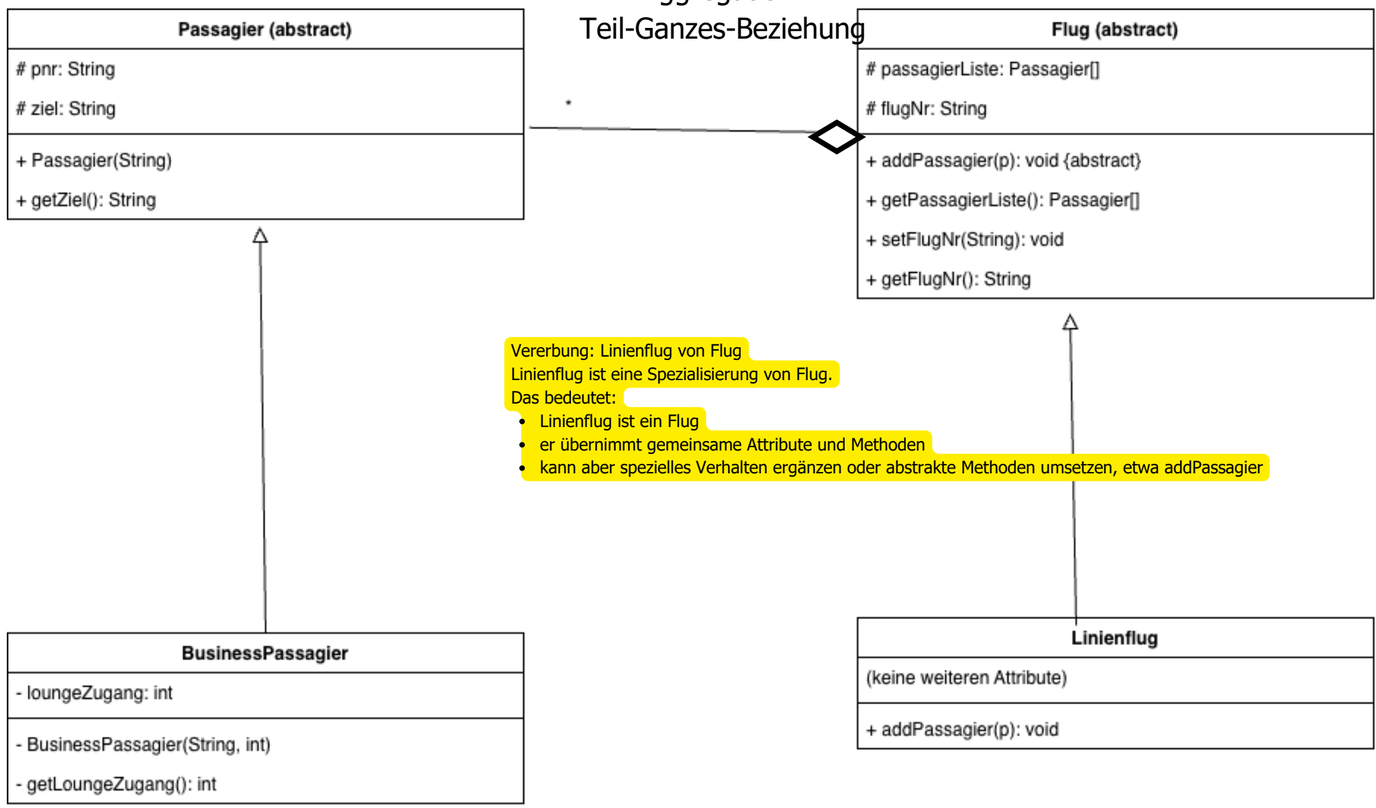
Im Flug gibt es eine passagierListe: Passagier[].
Das bedeutet: Ein Flug enthält mehrere Passagiere.
Das ist eine „hat-ein"-Beziehung.
Warum Aggregation?
Weil Passagiere auch unabhängig vom konkreten Flug als Objekte betrachtet werden können. Sie werden nicht vollständig durch den Flug „besessen", sondern sind darin enthalten.

Lösung

Fehlende Beziehungen:

1. Klasse 'Passagier': Aggregation zur Klasse 'Flug' (Klasse 'Passagier' ist Teil von Klasse 'Flug' – passagierListe in Flug ist vom Typ Passagier[])
2. Klasse 'BusinessPassagier': Vererbung an Klasse 'Passagier' (BusinessPassagier extends Passagier)
3. Klasse 'Linienflug': Vererbung an Klasse 'Flug' (Linienflug extends Flug)

Aggregation:
Teil-Ganzes-Beziehung



Vererbung: BusinessPassagier von Passagier
BusinessPassagier ist eine Spezialisierung von Passagier.
Das bedeutet:
- BusinessPassagier ist ein Passagier
- er erbt gemeinsame Eigenschaften wie pnr oder ziel
- zusätzlich besitzt er eigene Merkmale wie loungeZugang

Das ist typisch für objektorientierte Modellierung:
Allgemeines in die Oberklasse, Besonderes in die Unterklasse.

Vererbung: Linienflug von Flug
Linienflug ist eine Spezialisierung von Flug.
Das bedeutet:
- Linienflug ist ein Flug
- er übernimmt gemeinsame Attribute und Methoden
- kann aber spezielles Verhalten ergänzen oder abstrakte Methoden umsetzen, etwa addPassagier

CLOUDHELDEN.ORG

4.3 Konstruktor implementieren

Ihre Aufgabe:

Implementieren Sie im Pseudocode den Konstruktor der Klasse 'BusinessPassagier'.

Passagier (abstract)
pnr: String # ziel: String
+ Passagier(String) + getZiel(): String

Flug (abstract)
passagierListe: Passagier[] # flugNr: String
+ addPassagier(p): void {abstract} + getPassagierListe(): Passagier[] + setFlugNr(String): void + getFlugNr(): String

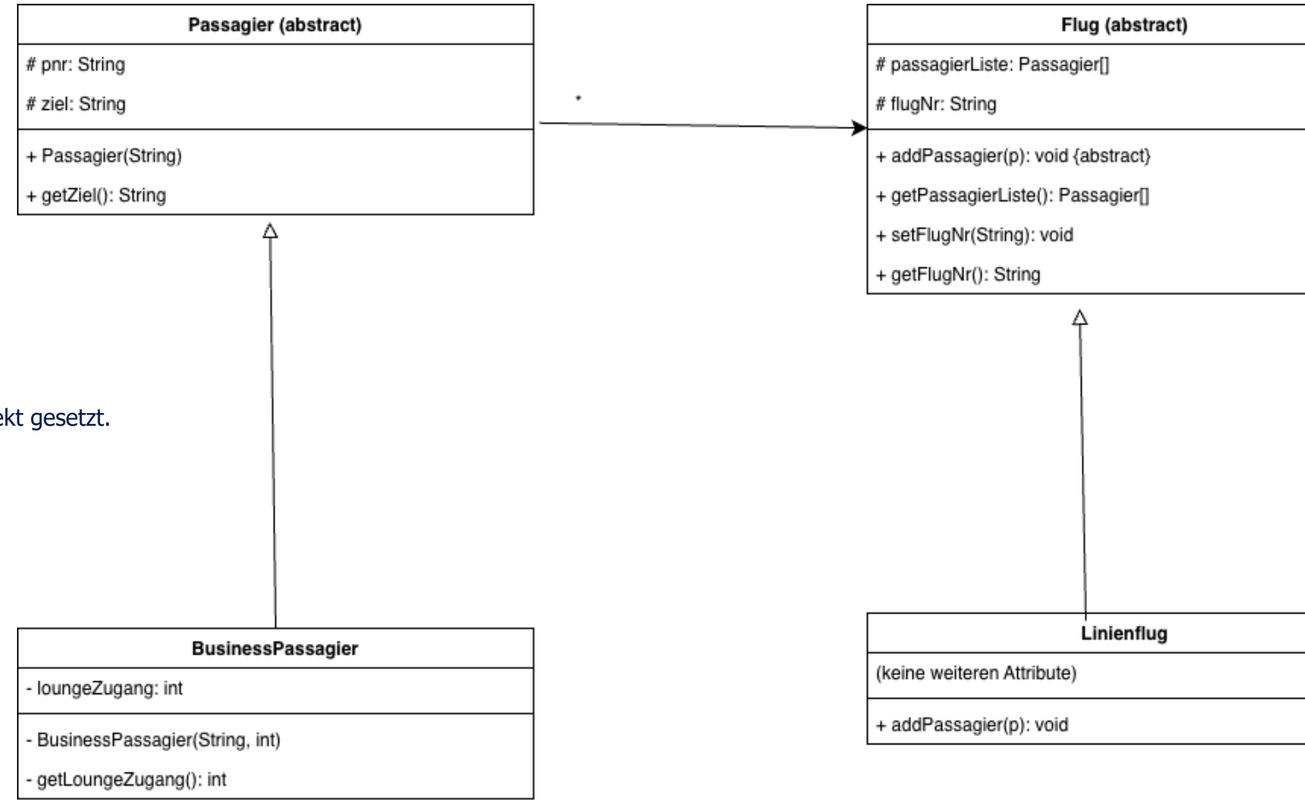
BusinessPassagier
- loungeZugang: int
- BusinessPassagier(String, int) - getLoungeZugang(): int

Linienflug
(keine weiteren Attribute)
+ addPassagier(p): void

Lösung

Musterlösung

```
public BusinessPassagier(String ziel, int loungeZugang) {
    super(ziel)
    this.loungeZugang = loungeZugang
}
```



super(ziel): Damit wird der Konstruktor der Oberklasse Passagier aufgerufen. So wird das geerbte Attribut ziel korrekt gesetzt.

- Warum ist das wichtig?
  - Weil ziel nicht in BusinessPassagier, sondern in Passagier definiert ist.

this.loungeZugang = loungeZugang

Hier wird der übergebene Parameter in das Attribut der aktuellen Klasse gespeichert.

Warum this?

- Weil Parametername und Attributname gleich heißen.
- this.loungeZugang meint das Attribut des Objekts,
- loungeZugang rechts meint den übergebenen Wert.

Unterklassen initialisieren:

- geerbte Attribute über super(...)
- eigene Attribute über this...

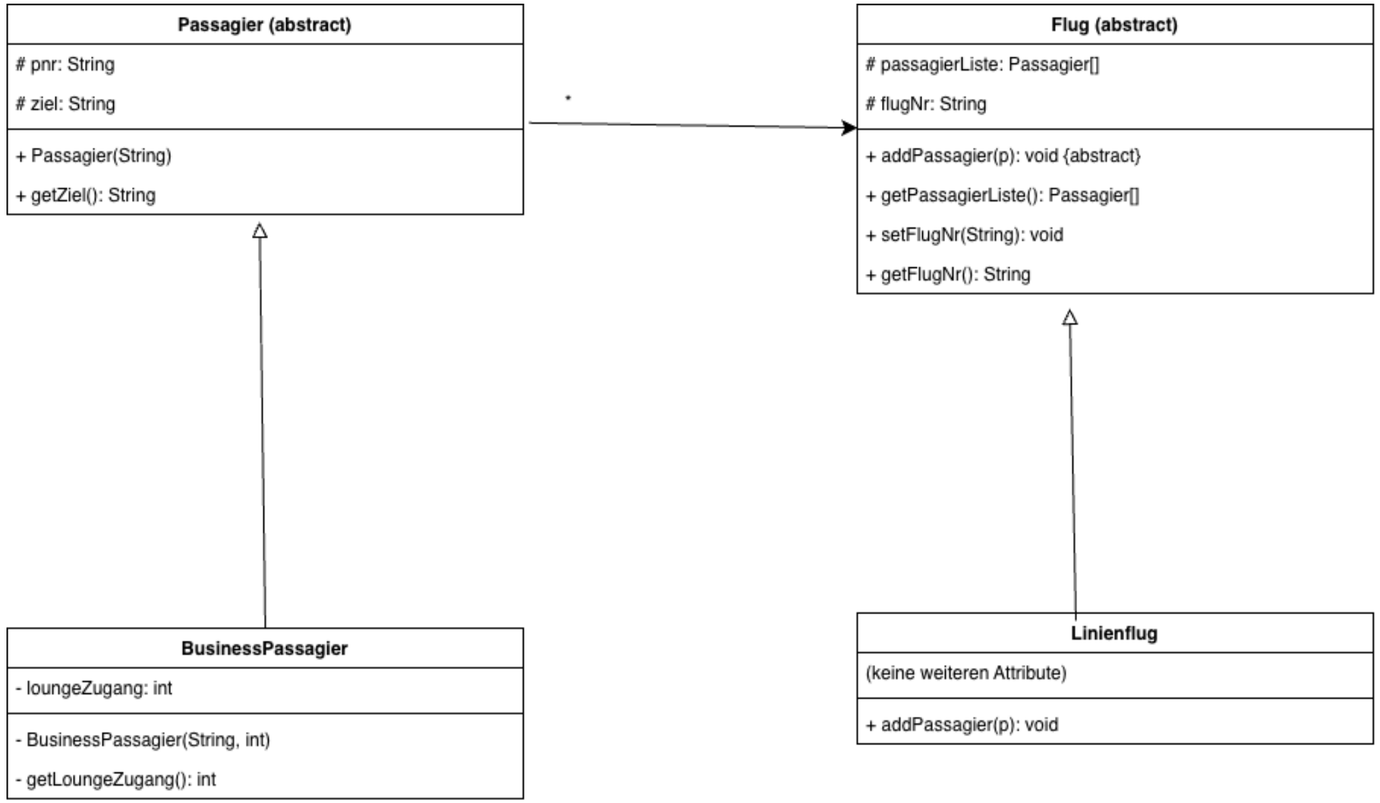
CLOUDHELDEN.ORG

4.4 Methode implementieren

Ihre Aufgabe:

Implementieren Sie im Pseudocode die Methode 'addPassagier' der Klasse 'Linienflug'.

Ein neuer Passagier soll dabei an die erste freie Stelle in der passagierListe eingefügt werden. Handelt es sich um den ersten Passagier ohne Eintrag, soll dieser an die erste verfügbare Position eingefügt werden.



CLOUDHELDEN.ORG

Lösung

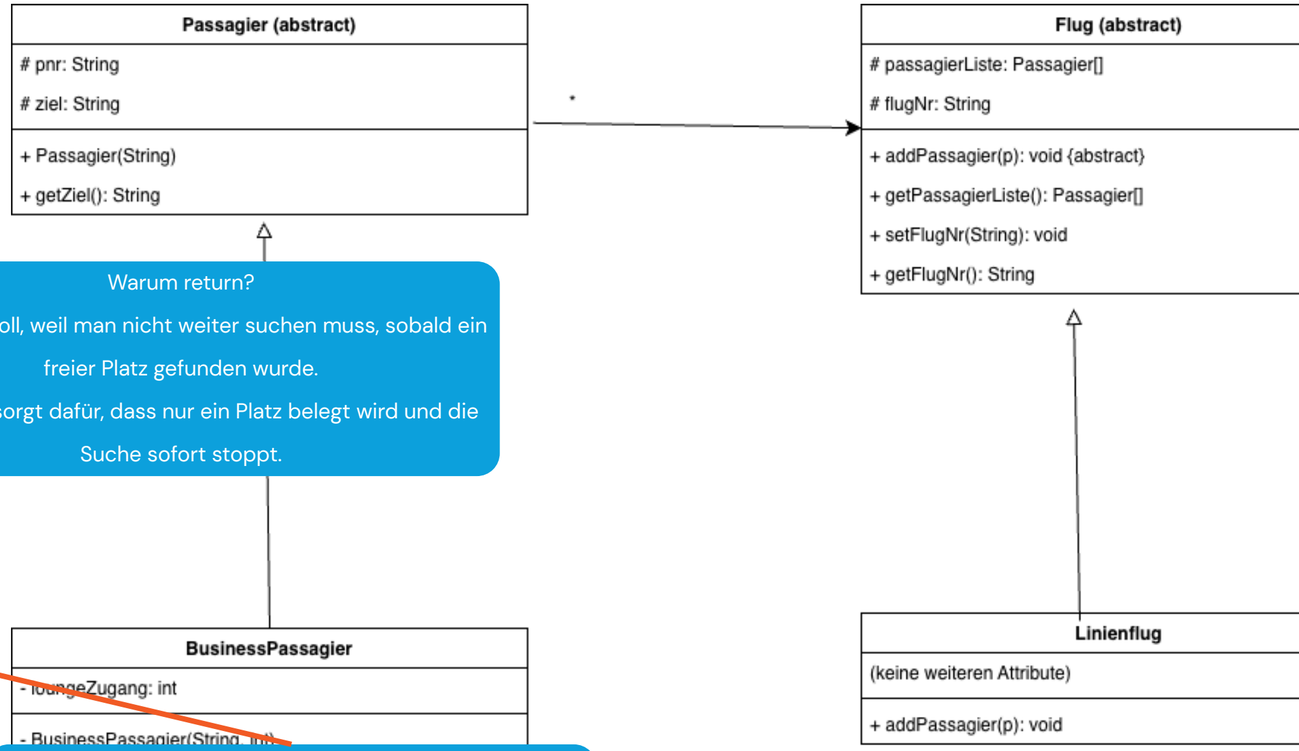
Musterlösung

Ein neuer Passagier soll dabei an die erste freie Stelle in der passagierListe eingefügt werden. Handelt es sich um den ersten Passagier ohne Eintrag, soll dieser an die erste verfügbare Position eingefügt werden.

```
public void addPassagier(Passagier passagier) {
    wenn (passagier == null) {
        return
    }
    für (i = 0; i < passagierListe.length - 1; i++) {
        wenn (passagierListe[i] == null) {
            passagierListe[i] = passagier
            return
        }
    }
    // Fallback: letzten Platz belegen
    passagierListe[passagierListe.length - 1] = passagier
}
```

- Die Null-Prüfung am Anfang ist essenziell, um NullPointerExceptions zu vermeiden.
- Die Schleife iteriert durch das Array und sucht die erste leere (null) Stelle.
- Der Fallback am Ende (letzter Platz) greift, wenn alle Plätze belegt sind.
- Das 'return' innerhalb der Schleife sorgt für eine effiziente Beendigung nach dem Einfügen.

Achtung: Das wurde mir den letzten Passagier immer überschreiben. Beachte den realen Kontext, wenn wir auf Überbuchungen achten müssen, hätten viele Leute vllt denselben Sitzplatz zugeordnet, obwohl wir gar nicht den Platz



Warum return?

Das ist sinnvoll, weil man nicht weiter suchen muss, sobald ein freier Platz gefunden wurde.

Das return sorgt dafür, dass nur ein Platz belegt wird und die Suche sofort stoppt.

Fallback auf den letzten Platz

Die Musterlösung setzt am Ende den letzten Platz.

Das ist etwas pragmatisch formuliert. Fachlich bedeutet es: Falls die Schleife keine freie Stelle gefunden hat, wird noch der letzte Index beschrieben.

Streng genommen wäre das nur dann sinnvoll, wenn der letzte Platz eventuell noch nicht geprüft wurde. Sonst könnte man damit einen vorhandenen Eintrag überschreiben. In einer echten Implementierung wäre es sauberer, bei vollständig belegtem Array entweder:
nichts zu tun,
einen Fehler zu werfen,
oder die Liste zu vergrößern.

CLOUDHELDEN.ORG

Kahooooot!